

Ex. No. : 07

Date:

Register No.: 231701016

Name: Harish A

A PYTHON PROGRAM TO IMPLEMENT DECISION TREE

Aim:

To implement a **Decision Tree model in Python** to classify **fraudulent and normal credit card transactions**.

Requirements:

Jupyter Notebook with Python, creditcard.csv dataset, and required Python libraries.

PROGRAM:

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
import matplotlib.pyplot as plt
from sklearn import tree
```

Step 2: Upload Dataset

```
from google.colab import files
uploaded = files.upload()
```

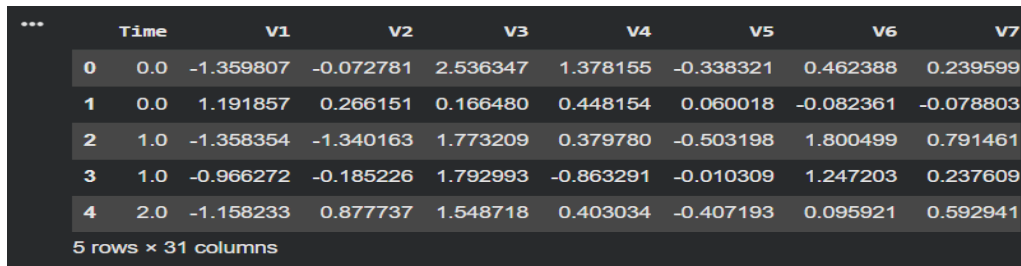
Upload:

creditcard.csv

Step 3: Load the Dataset

```
data = pd.read_csv("creditcard.csv")
```

```
data.head()
```



	Time	V1	V2	V3	V4	V5	V6	V7
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941

5 rows x 31 columns

Step 4: Define Features and Target

Target column:

Class

0 → Normal Transaction

1 → Fraud Transaction

```
X = data.drop("Class", axis=1)
```

```
y = data["Class"]
```

Step 5: Train-test split

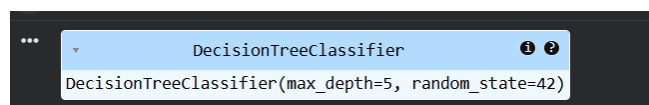
```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)
```

Step 6: Create Decision Tree model

```
model = DecisionTreeClassifier(max_depth=5, random_state=42)
```

Step 7: Train model

```
model.fit(X_train, y_train)
```



Step 8: Predict the Results

```
y_pred = model.predict(X_test)
```

Step 9: Evaluate the Model

Accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.999553303916762
```

Confusion Matrix

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Confusion Matrix:
[[91618   9]
 [  32 126]]
```

Classification Report

```
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

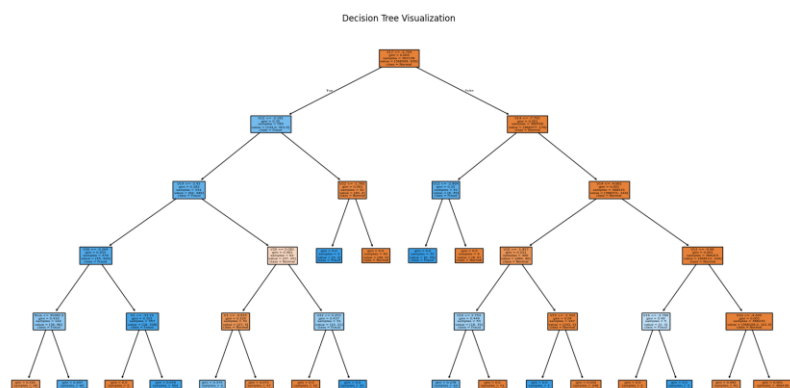
```
Classification Report:
              precision    recall  f1-score   support

     0.0       1.00      1.00      1.00     91627
     1.0       0.93      0.80      0.86       158

 accuracy      0.97      0.90      0.93     91785
  macro avg           0.97      0.90      0.93     91785
 weighted avg           1.00      1.00      1.00     91785
```

Step 10: Visualize Decision Tree

```
plt.figure(figsize=(20,10))
tree.plot_tree(model, filled=True, feature_names=X.columns,
class_names=['Normal','Fraud'])
plt.title("Decision Tree Visualization")
plt.show()
```



Result:

Thus, the **Decision Tree Classifier** was successfully implemented using Python and evaluated using the Credit Card Fraud Detection dataset.